



Mediacrest Training College

Nairobi, Kenya

Advanced Data Analytics & Machine Learning

8-Week Advanced Training Program

Comprehensive Session Notes

Week 5 · Days 2

Supervised Classification Algorithms: Logistic Regression, KNN, SVM, & Naive Bayes

Course Trainer: Victor Wandera Lumumba, MSc.

Role: Statistician, Data Analyst & ML Expert

Session Date: September 2026

Mode: Hybrid – Online / Physical (EAT)

“Mathematical rigor is the foundation of trustworthy AI. Understanding the likelihood, margins, and probabilities behind the code is what separates a technician from a true data scientist.”

About This Document

These comprehensive notes accompany **Week 5** of the **Advanced Data Analytics & Machine Learning** program at Mediacrest Training College. This week pivots from regression to **Supervised Classification**, providing deep mathematical and practical foundations for four cornerstone algorithms: Logistic Regression, K-Nearest Neighbors (KNN), Support Vector Machines (SVM), and Naive Bayes.

Building on the ‘Heart.csv’ dataset, these notes bridge the gap between abstract mathematical theory (likelihood functions, optimization, kernel tricks) and applied clinical decision-making in the Kenyan healthcare context.

Course at a Glance

	8 Weeks (20+ intensive sessions)
Duration	
Core Tools	Python (pandas, NumPy, scikit-learn, statsmodels, matplotlib, seaborn)
Focus	Mathematical foundations, algorithmic assumptions, hyperparameter tuning, clinical evaluation
Capstone	Production-ready ML system with diagnostics, explainability & policy recommendations
Context	Healthcare, Clinical Risk Prediction, and Epidemiological Modeling

How to Use These Notes

- **Master the Math:** The **Mathematical Foundation** boxes contain derivations essential for understanding *why* algorithms behave the way they do.
- **Follow along in code:** Every listing is runnable. Open the companion Jupyter Notebook and execute each cell as you read.
- **Contextualize:** **Kenyan Clinical Context** boxes connect abstract math to local healthcare realities (e.g., MOH policy, resource constraints).

Contents

About This Document	1
1 Foundations of Classification & The Heart Dataset	3
1.1 The Classification Paradigm	3
1.2 The Heart Disease Dataset ('Heart.csv')	3
2 Logistic Regression: The Statistical Baseline	3
2.1 From Linear to Logistic	3
2.2 Maximum Likelihood Estimation (MLE)	4
2.3 Optimization: Gradient Descent	4
2.4 Regularization: Preventing Overfitting	4
3 K-Nearest Neighbors (KNN): Instance-Based Learning	5
3.1 The Core Philosophy	5
3.2 The Curse of Dimensionality	5
3.3 Critical Assumptions & Preprocessing	5
3.4 Weighted KNN	6
4 Support Vector Machines (SVM): Maximizing the Margin	6
4.1 The Maximal Margin Classifier	6
4.2 Soft Margin and Hinge Loss	6
4.3 The Kernel Trick	7
5 Naive Bayes: Probabilistic Classification	7
5.1 Bayes' Theorem	7
5.2 The "Naive" Assumption	7
5.3 Gaussian Naive Bayes for Continuous Features	8
5.4 Laplace Smoothing	8
6 Model Evaluation & Clinical Trade-offs	8
6.1 Beyond Accuracy: The Confusion Matrix	8
6.2 ROC Curve and AUC	9
7 Practical Implementation on 'Heart.csv'	9
7.1 Unified Preprocessing Pipeline	9
7.2 Model Definition & Training	10
8 Summary & Policy Recommendations	11
8.1 Algorithm Selection Guide for Clinical Data	11
9 Glossary of Key Mathematical Terms	11
10 References & Further Reading	12

1 Foundations of Classification & The Heart Dataset

1.1 The Classification Paradigm

Unlike regression, which predicts a continuous quantity $y \in \mathbb{R}$, classification aims to predict a discrete class label $y \in \{C_1, C_2, \dots, C_k\}$. In binary classification, $y \in \{0, 1\}$.

The fundamental goal is to learn a decision function $f: \mathcal{X} \rightarrow \mathcal{Y}$ or a conditional probability distribution $P(Y|X)$ from a labeled training dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$.

1.2 The Heart Disease Dataset ('Heart.csv')

This dataset contains clinical cardiovascular measurements for patients.

- **Target Variable:** HD (0 = No Heart Disease, 1 = Heart Disease).
- **Numeric Features:** Age, RestBP (Resting Blood Pressure), Chol (Serum Cholesterol), MaxHR (Maximum Heart Rate), Oldpeak (ST depression induced by exercise).
- **Categorical Features:** Sex, ChestPain (typical, asymptomatic, etc.), Thal (thalassemia: normal, fixed, reversible).

Kenyan Clinical Context

In Kenyan county hospitals, non-communicable diseases (NCDs) like cardiovascular disease are rising. A robust classification model can act as a triage tool, helping Community Health Promoters (CHPs) identify high-risk individuals for early referral to facilities like Kenyatta National Hospital (KNH) or Moi Teaching and Referral Hospital (MTRH), optimizing limited specialist resources.

2 Logistic Regression: The Statistical Baseline

2.1 From Linear to Logistic

Linear regression is unsuitable for classification because it can output values outside $[0, 1]$, which cannot be interpreted as probabilities. Logistic regression models the **log-odds** (logit) of the probability as a linear combination of features.

The Logit Function

Let $p = P(Y = 1|X = x)$. The odds of the event are $\frac{p}{1-p}$. The logit function is:

$$\text{logit}(p) = \ln\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n = \beta^T x$$

Solving for p , we obtain the **Sigmoid (Logistic) Function**:

$$p = \sigma(\beta^T x) = \frac{1}{1 + e^{-\beta^T x}}$$

2.2 Maximum Likelihood Estimation (MLE)

Unlike Ordinary Least Squares (OLS) in linear regression, Logistic Regression parameters β are estimated using Maximum Likelihood Estimation. We want to find the β that makes the observed data most probable.

Likelihood and Log-Likelihood Derivation

For a single observation, the probability can be written compactly as:

$$P(y_i|x_i, \beta) = p_i^{y_i} (1 - p_i)^{1-y_i}$$

Assuming independent observations, the **Likelihood Function** $L(\beta)$ for the entire dataset is:

$$L(\beta) = \prod_{i=1}^N p_i^{y_i} (1 - p_i)^{1-y_i}$$

To simplify optimization (and prevent underflow), we take the natural logarithm to get the **Log-Likelihood Function** $\ell(\beta)$:

$$\ell(\beta) = \sum_{i=1}^N [y_i \ln(p_i) + (1 - y_i) \ln(1 - p_i)]$$

Substituting $p_i = \sigma(\beta^T x_i)$, the objective is to **maximize** $\ell(\beta)$ with respect to β . In machine learning, we typically minimize the **Negative Log-Likelihood (NLL)**, also known as Binary Cross-Entropy Loss:

$$J(\beta) = -\frac{1}{N} \sum_{i=1}^N [y_i \ln(\sigma(\beta^T x_i)) + (1 - y_i) \ln(1 - \sigma(\beta^T x_i))]$$

2.3 Optimization: Gradient Descent

There is no closed-form solution (like the Normal Equation in OLS) for Logistic Regression. We use iterative optimization, typically Gradient Descent. The gradient of the cost function $J(\beta)$ with respect to β is elegantly simple:

$$\nabla_{\beta} J(\beta) = \frac{1}{N} \sum_{i=1}^N (\sigma(\beta^T x_i) - y_i) x_i = \frac{1}{N} X^T (\hat{y} - y)$$

We update the weights iteratively: $\beta^{(t+1)} = \beta^{(t)} - \alpha \nabla_{\beta} J(\beta)$, where α is the learning rate.

2.4 Regularization: Preventing Overfitting

In high-dimensional clinical data, features may be highly correlated (multicollinearity), causing β coefficients to explode. We add a penalty term to the loss function.

- **L2 Regularization (Ridge):** $J(\beta) + \lambda \sum_{j=1}^n \beta_j^2$. Shrinks coefficients toward zero, improving stability.
- **L1 Regularization (Lasso):** $J(\beta) + \lambda \sum_{j=1}^n |\beta_j|$. Forces some coefficients to exactly zero, performing automatic feature selection.

The Odds Ratio vs. Risk Ratio Trap

Logistic regression outputs Odds Ratios (OR). If the baseline prevalence of heart disease is high (e.g., 50%), the OR will **grossly exaggerate** the true Risk Ratio (RR). Always convert OR to marginal predicted probabilities for patient-facing communication in Kenyan clinical settings.

3 K-Nearest Neighbors (KNN): Instance-Based Learning

3.1 The Core Philosophy

KNN is a **non-parametric, lazy learning** algorithm. It makes no assumptions about the underlying data distribution. Instead, it memorizes the training dataset and classifies a new query point x_q based on the majority class of its k closest neighbors in the feature space.

Distance Metrics

The definition of "closest" depends on the distance metric $d(x, x')$. For two n -dimensional vectors x and x' :

1. **Euclidean Distance (L2 Norm):** $d(x, x') = \sqrt{\sum_{j=1}^n (x_j - x'_j)^2}$
2. **Manhattan Distance (L1 Norm):** $d(x, x') = \sum_{j=1}^n |x_j - x'_j|$
3. **Minkowski Distance:** $d(x, x') = \left(\sum_{j=1}^n |x_j - x'_j|^p\right)^{1/p}$ (Euclidean is $p = 2$, Manhattan is $p = 1$).

3.2 The Curse of Dimensionality

Mathematical Intuition of the Curse

As the number of features n increases, the volume of the feature space grows exponentially. Consequently, data points become sparse. Mathematically, the ratio of the distance to the nearest neighbor to the distance to the farthest neighbor approaches 1:

$$\lim_{n \rightarrow \infty} \frac{d_{max} - d_{min}}{d_{min}} \rightarrow 0$$

This means **all points become roughly equidistant**, rendering distance metrics meaningless and causing KNN performance to degrade severely.

3.3 Critical Assumptions & Preprocessing

Mandatory Feature Scaling

Because KNN relies entirely on distance calculations, features with larger magnitudes (e.g., **Chol** in the 200s) will completely dominate features with smaller magnitudes (e.g., **Ca** in the 0-3 range). **Standardization (Z-score scaling)** is **strictly mandatory** before applying KNN.

3.4 Weighted KNN

To mitigate the impact of distant neighbors within the k -radius, we can assign weights inversely proportional to the distance:

$$w_i = \frac{1}{d(x_q, x_i)}$$

The predicted class is the one that maximizes the sum of weights for that class, rather than a simple majority vote.

4 Support Vector Machines (SVM): Maximizing the Margin

4.1 The Maximal Margin Classifier

SVM aims to find the optimal hyperplane that separates the two classes with the maximum possible margin. A hyperplane in n -dimensional space is defined by:

$$w^T x + b = 0$$

where w is the weight vector (normal to the hyperplane) and b is the bias.

Hard Margin Optimization

For linearly separable data with labels $y_i \in \{-1, 1\}$, we require that all points satisfy:

$$y_i(w^T x_i + b) \geq 1 \quad \forall i$$

The margin width is $\frac{2}{\|w\|}$. To maximize the margin, we must minimize $\|w\|$. To make the optimization problem convex and differentiable, we minimize $\frac{1}{2}\|w\|^2$. **Primal Problem:**

$$\min_{w,b} \frac{1}{2}\|w\|^2 \quad \text{subject to} \quad y_i(w^T x_i + b) \geq 1 \quad \forall i$$

4.2 Soft Margin and Hinge Loss

Real-world clinical data (like 'Heart.csv') is rarely perfectly linearly separable. We introduce **slack variables** $\xi_i \geq 0$ to allow some misclassifications, controlled by a regularization parameter C .

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C \sum_{i=1}^N \xi_i$$

subject to $y_i(w^T x_i + b) \geq 1 - \xi_i$.

This is equivalent to minimizing the **Hinge Loss**:

$$J(w) = \frac{1}{2}\|w\|^2 + C \sum_{i=1}^N \max(0, 1 - y_i(w^T x_i + b))$$

- **Large C :** Low bias, high variance (strictly penalizes misclassifications, risks overfitting).
- **Small C :** High bias, low variance (wider margin, allows more misclassifications, better generalization).

4.3 The Kernel Trick

When data is not linearly separable in the original feature space, SVM maps it to a higher-dimensional space using a transformation $\phi(x)$, where it may become linearly separable. Instead of computing $\phi(x)$ explicitly (which is computationally prohibitive), we use a **Kernel Function** $K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$.

Common Kernel Functions

1. **Linear Kernel:** $K(x_i, x_j) = x_i^T x_j$ (Equivalent to standard Logistic Regression/SVM).
2. **Polynomial Kernel:** $K(x_i, x_j) = (\gamma x_i^T x_j + r)^d$
3. **Radial Basis Function (RBF) / Gaussian Kernel:** $K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$.
This is the most powerful and commonly used kernel, creating complex, non-linear decision boundaries.

5 Naive Bayes: Probabilistic Classification

5.1 Bayes' Theorem

Naive Bayes is a generative probabilistic model based on Bayes' Theorem:

$$P(Y = y|X = x) = \frac{P(X = x|Y = y)P(Y = y)}{P(X = x)}$$

Since $P(X = x)$ is constant for all classes, we can simplify the classification rule to Maximum A Posteriori (MAP) estimation:

$$\hat{y} = \arg \max_y P(Y = y) \prod_{j=1}^n P(X_j = x_j|Y = y)$$

5.2 The "Naive" Assumption

The algorithm is called "Naive" because it assumes **conditional independence** between all features X_j given the class Y .

$$P(X_1, X_2, \dots, X_n|Y) = \prod_{j=1}^n P(X_j|Y)$$

Clinical Reality Check

In medicine, features are rarely independent (e.g., **RestBP** and **Chol** are correlated). Despite this violated assumption, Naive Bayes often performs surprisingly well in practice, especially with high-dimensional data, because it only needs to estimate the correct *ranking* of probabilities, not the exact probabilities.

5.3 Gaussian Naive Bayes for Continuous Features

For continuous clinical variables like **Age** or **MaxHR**, we assume they follow a Gaussian (Normal) distribution within each class y .

$$P(X_j = x_j | Y = y) = \frac{1}{\sqrt{2\pi\sigma_{y,j}^2}} \exp\left(-\frac{(x_j - \mu_{y,j})^2}{2\sigma_{y,j}^2}\right)$$

where $\mu_{y,j}$ and $\sigma_{y,j}^2$ are the mean and variance of feature j for class y , estimated directly from the training data.

5.4 Laplace Smoothing

For categorical features, if a category appears in the test set but not in the training set for a specific class, the probability becomes 0, wiping out the entire product. We apply **Laplace (Add-One) Smoothing**:

$$P(X_j = x_j | Y = y) = \frac{\text{count}(x_j, y) + 1}{\text{count}(y) + V}$$

where V is the number of unique categories for feature j .

6 Model Evaluation & Clinical Trade-offs

6.1 Beyond Accuracy: The Confusion Matrix

Accuracy is dangerously misleading in imbalanced clinical datasets. We must decompose predictions into four quadrants:

- **True Positive (TP):** Model predicts Disease, Patient has Disease.
- **False Positive (FP):** Model predicts Disease, Patient is Healthy. (Cost: Unnecessary, expensive angiogram, patient anxiety).
- **False Negative (FN):** Model predicts Healthy, Patient has Disease. (Cost: Missed diagnosis, potential mortality).
- **True Negative (TN):** Model predicts Healthy, Patient is Healthy.

Derived Metrics

$$\text{Precision (PPV)} = \frac{TP}{TP + FP} \quad (\text{Of all flagged, how many are truly sick?})$$

$$\text{Recall (Sensitivity)} = \frac{TP}{TP + FN} \quad (\text{Of all truly sick, how many did we catch?})$$

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (\text{Harmonic mean, balances both})$$

6.2 ROC Curve and AUC

The Receiver Operating Characteristic (ROC) curve plots the True Positive Rate (Recall) against the False Positive Rate ($1 - \text{Specificity}$) at various classification thresholds.

$$\text{FPR} = \frac{FP}{FP + TN}$$

The **Area Under the Curve (AUC)** represents the probability that the model will rank a randomly chosen positive instance higher than a randomly chosen negative instance. An AUC of 0.5 is random guessing; 1.0 is perfect.

Threshold Selection for Policy

- **Primary Care Screening (CHPs):** Prioritize **Recall**. Lower the threshold (e.g., 0.3). We accept more FPs to ensure no sick patient is missed (FNs are catastrophic).
- **Specialist Referral (Cardiology):** Prioritize **Precision**. Raise the threshold (e.g., 0.7). We must be highly confident before recommending an invasive, costly procedure at a referral hospital.

7 Practical Implementation on ‘Heart.csv’

7.1 Unified Preprocessing Pipeline

We must handle missing values, encode categoricals, and scale numerals to satisfy the assumptions of KNN, SVM, and Logistic Regression.

```

1 import pandas as pd
2 import numpy as np
3 from sklearn.model_selection import train_test_split
4 from sklearn.compose import ColumnTransformer
5 from sklearn.preprocessing import StandardScaler, OneHotEncoder
6 from sklearn.pipeline import Pipeline
7
8 # 1. Load Data
9 df = pd.read_csv('Heart.csv', na_values=['NA', '?', 'NaN', ''])
10
11 # 2. Imputation
12 df['Ca'] = pd.to_numeric(df['Ca'], errors='coerce').fillna(df['Ca'].median())
13 df['Thal'] = df['Thal'].fillna('unknown')
14
15 # 3. Define Features and Target
16 X = df.drop(columns=['HD', 'AHD']) # AHD is string duplicate of HD
17 y = df['HD']
18
19 # 4. Preprocessor
20 numeric_features = ['Age', 'RestBP', 'Chol', 'MaxHR', 'Oldpeak']
21 categorical_features = ['Sex', 'ChestPain', 'Fbs', 'RestECG', 'ExAng', 'Slope',
22                        'Ca', 'Thal']
23
24 preprocessor = ColumnTransformer(
25     transformers=[
26         ('num', StandardScaler(), numeric_features),

```

```
26         ('cat', OneHotEncoder(drop='first', handle_unknown='ignore'),
27         categorical_features)
28     ])
29 # 5. Train/Test Split (Stratified to maintain class balance)
30 X_train, X_test, y_train, y_test = train_test_split(
31     X, y, test_size=0.25, random_state=42, stratify=y
32 )
```

Listing 1: Robust Preprocessing Pipeline

7.2 Model Definition & Training

```
1 from sklearn.linear_model import LogisticRegression
2 from sklearn.neighbors import KNeighborsClassifier
3 from sklearn.svm import SVC
4 from sklearn.naive_bayes import GaussianNB
5 from sklearn.metrics import classification_report, roc_auc_score
6
7 # 1. Logistic Regression (with L2 regularization)
8 logreg_pipe = Pipeline([('prep', preprocessor),
9                          ('clf', LogisticRegression(random_state=42, max_iter
10                          =1000))])
11
12 # 2. K-Nearest Neighbors (k=5, Euclidean distance)
13 knn_pipe = Pipeline([('prep', preprocessor),
14                      ('clf', KNeighborsClassifier(n_neighbors=5, metric='
15                      minkowski', p=2))])
16
17 # 3. Support Vector Machine (RBF Kernel)
18 svm_pipe = Pipeline([('prep', preprocessor),
19                      ('clf', SVC(kernel='rbf', C=1.0, gamma='scale', probability
20                      =True, random_state=42))])
21
22 # 4. Gaussian Naive Bayes
23 nb_pipe = Pipeline([('prep', preprocessor),
24                     ('clf', GaussianNB())])
25
26 # Train all models
27 models = {'Logistic Regression': logreg_pipe, 'KNN': knn_pipe, 'SVM': svm_pipe,
28           'Naive Bayes': nb_pipe}
29
30 for name, model in models.items():
31     model.fit(X_train, y_train)
32     y_pred = model.predict(X_test)
33     y_prob = model.predict_proba(X_test)[:, 1]
34
35     print(f"--- {name} ---")
36     print(classification_report(y_test, y_pred))
37     print(f"ROC-AUC: {roc_auc_score(y_test, y_prob):.4f}\n")
```

Listing 2: Defining and Training the Four Algorithms

Pro Tip: SVM Probability

By default, scikit-learn's `SVC` does not output probabilities. You must set `probability=True`, which enables Platt scaling (a post-hoc logistic regression fitted on the SVM's decision scores) to generate calibrated probabilities for ROC-AUC calculation.

8 Summary & Policy Recommendations

8.1 Algorithm Selection Guide for Clinical Data

- **Logistic Regression:** Best for **interpretability**. When MOH stakeholders need to understand exactly how much `Chol` increases risk (via Odds Ratios), this is the gold standard.
- **KNN:** Best for **small, low-dimensional datasets** with clear local clustering. Avoid if the dataset has > 20 features due to the curse of dimensionality.
- **SVM:** Best for **complex, non-linear boundaries** in medium-sized datasets. The RBF kernel is highly powerful, but the model is a "black box" and training time scales poorly ($O(N^2)$ to $O(N^3)$) with large datasets.
- **Naive Bayes:** Best as a **fast, robust baseline**, especially with high-dimensional categorical data. Its independence assumption is a weakness, but its speed and resistance to overfitting are strengths.

Capstone Preparation: The Algorithmic Audit

Imagine you are deploying the SVM model to a county hospital. The hospital administrator asks: *"Why did the model flag this 45-year-old female patient as high risk?"*

1. Explain why SVM cannot answer this directly (lack of intrinsic feature importance).
2. Propose a solution using Explainable AI (XAI) techniques (e.g., SHAP values or LIME) to be covered in Week 7.
3. Draft a 2-sentence response to the administrator that balances technical honesty with clinical reassurance.

9 Glossary of Key Mathematical Terms

Binary Cross-Entropy The negative log-likelihood loss function used to train logistic regression, measuring the divergence between predicted probabilities and true labels.

Curse of Dimensionality The phenomenon where the volume of the feature space grows exponentially with the number of features, causing data sparsity and rendering distance metrics ineffective.

Hinge Loss The loss function used in SVMs, defined as $\max(0, 1 - y_i f(x_i))$, which penalizes misclassifications and points within the margin.

- Kernel Trick** A method in SVMs that computes the dot product of data points in a high-dimensional space without explicitly calculating the coordinates in that space, using a kernel function $K(x_i, x_j)$.
- Maximum Likelihood Estimation (MLE)** A method of estimating the parameters of a statistical model by finding the parameter values that maximize the likelihood of making the observations given the parameters.
- Naive Assumption** The assumption in Naive Bayes that all features are conditionally independent of each other given the class label.

10 References & Further Reading

- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer. (Chapters 4 & 7 for Logistic Regression and SVM).
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning*. Springer. (The definitive mathematical text on KNN, SVM, and Naive Bayes).
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning with Applications in Python*. Springer.
- Mediacrest Training College. (2026). *Advanced Data Analytics & Machine Learning Curriculum*.
- Murphy, K. P. (2012). *Machine Learning: A Probabilistic Perspective*. MIT Press. (For rigorous Naive Bayes and MLE derivations).

Mediacrest Training College – Nairobi, Kenya
Advanced Data Analytics & Machine Learning · Week 5 Comprehensive Notes
Trainer: Victor Wandera Lumumba, MSc. · September 2026
